Summer Internship Technical Report

Name: Ajay Venkatesh

Program: M.S., Computer Engineering, New York University

Internship Host: Amazon – AFT Inbound (Pallet CV Console)

Duration: Summer 2025

1. Introduction

During Summer 2025, I interned with Amazon's AFT (Amazon Fulfillment Technologies) Inbound organization on the Pallet CV Console—a web application that provides visual evidence and event metadata for pallets moving through fulfillment centers. The system integrates two vision subsystems: Pallet CV camera towers at dock doors and pallet staging areas, and PAD 1.2 ceiling-mounted cameras that capture overhead imagery. Together, these subsystems detect pallet movement, decode barcodes, surface defects (e.g., broken pallets, missing labels), and persist images, videos, and metadata for downstream analysis. My internship focused on re-architecting the Pallet CV Console to be enterprise-ready with robust authentication and fine-grained authorization while maintaining and improving its UX for FC associates, stakeholders, and developers.

Context and problem space: Fulfillment Centers (FCs) ingest thousands of pallets daily across dock doors and staging areas. Operations teams need fast, trustworthy visual evidence for each pallet (what entered, when, where, and in what condition) to resolve defects and disputes. Before this project, evidence existed but was difficult to audit end-to-end, and access was not constrained by role or site. The Pallet CV Console aims to unify evidence (images/videos + event metadata) and make access safe, auditable, and fast for the right users.

Users and value: Primary users are FC Associates and Process Assistants who investigate dock events; stakeholders include Program/Tech Managers who analyze trends, and developers who validate vision pipelines. The console reduces mean-time-to-insight (MTTI) by pairing timeline metadata with media, standardizing orientation/ordering, and enabling direct media access where appropriate. For leadership, the system increases trust by enforcing least privilege and capturing audit trails for access decisions.

Data and flow summary: Vision subsystems (Pallet CV towers and PAD 1.2 overhead cameras) generate event metadata and media references stored in DynamoDB and S3. The console authenticates users through Midway SSO on Harmony and authorizes via

Helis (role × FC). Authorized requests query DynamoDB for event timelines and return presigned S3 URLs for media display. My work centered on hardening this path—defining the identity/authorization contract, tuning event queries, and fixing media-orientation/ordering to make investigations dependable.

My contribution in brief: I re-platformed the app on Harmony, implemented Midway SSO and Helis-backed authorization, refined API contracts (Smithy), compared CBOR vs. JSON for payload efficiency, and addressed image handling and UI ergonomics. I also documented runbooks and a demo path showing distinct experiences for DEV, Stakeholder, and FC Associate, including a deliberate "no-access" case.

2. Relationship to My Academic Program

This internship connected directly to my courses—especially Java. I implemented Java 21 Lambda with Coral and Smithy, used Dagger for dependency injection and immutable DTOs for safe boundaries, and relied on collections/generics, streams, and Optional for null-safety. I designed structured exceptions and clear error handling around Helis, DynamoDB, and S3; negotiated CBOR vs. JSON to improve latency; wrote unit and integration tests against API Gateway; and kept authentication/authorization cleanly separated from data retrieval.

Course-to-internship mapping (focused on what I actually took):

- Java → Practical application in production code: Lambda handlers, DI with Dagger, DTO validation, testability, and readable OOP structure.
- Computing Systems Architecture → Guided memory/CPU sizing for Lambdas, awareness of JVM cold-starts and object allocation, and the (de)serialization cost trade-offs that informed CBOR vs. JSON.
- Big Data → Shaped DynamoDB modeling (partition/sort keys, pagination) and S3 object layout for high-volume media; emphasized throughput, hot-partition avoidance, and cost-aware access patterns.

Across these subjects, Java was the implementation bridge—from design decisions down to reliable handlers and tests—turning classroom knowledge about systems and data into production-grade services for authorization and evidence retrieval.

3. Project Overview and Objectives

The existing console originated as a developer tool: it authenticated users but did not differentiate access based on role or FC (fulfillment center). This created unrestricted data visibility and limited enterprise readiness. The internship objectives were to:
• Re-platform the frontend onto Amazon's Harmony application platform to accelerate development and integrate with internal identity systems.
• Establish robust authentication and authorization to enforce least-privilege access by role and by FC.
• Preserve and enhance core product capabilities like image/video visualization, event details, and filtering, while improving image rotation/ordering behavior.
• Deliver a demoable, production-minded slice covering end-to-end flows from login to authorized data access.

3.1 Architecture Diagram

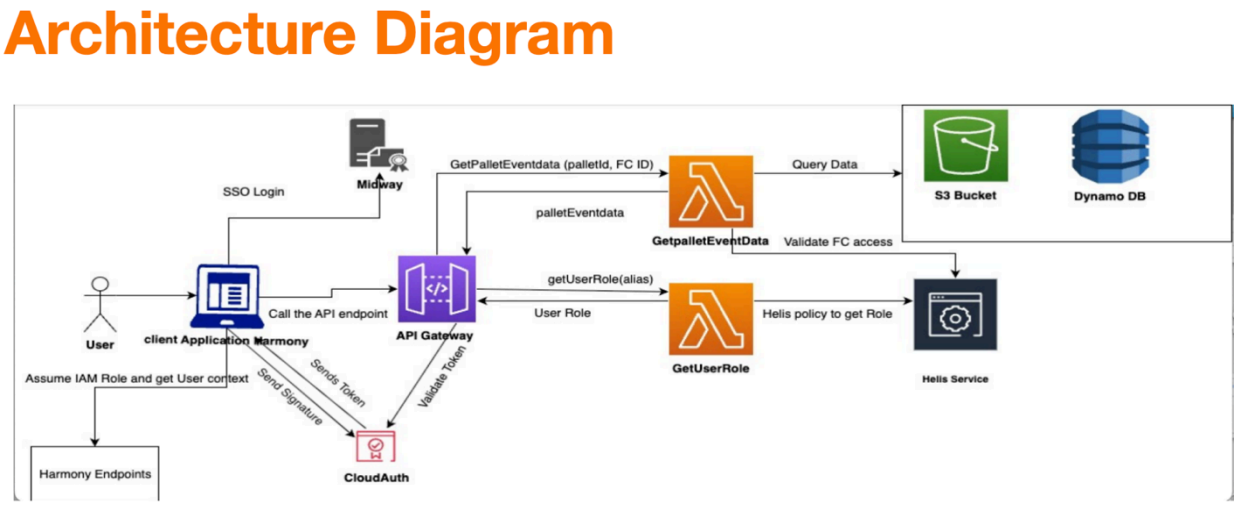

Figure 1: High-level architecture showing Harmony + Midway SSO, API Gateway, Helis-backed role/FC check, DynamoDB query, and S3 media access.

The architecture centers on Harmony (frontend), Midway (SSO), API Gateway, and two Lambda activities. The GetUserRole activity consults Helis to resolve the user's application role and FC POSIX groups. The GetPalletEventData activity validates that role/FC mapping before querying DynamoDB for event timelines and returning S3 media references (with presigned access when applicable). This enforces site-scoped least privilege while keeping the media path efficient for investigators.

3.2 Web Sequence Diagram

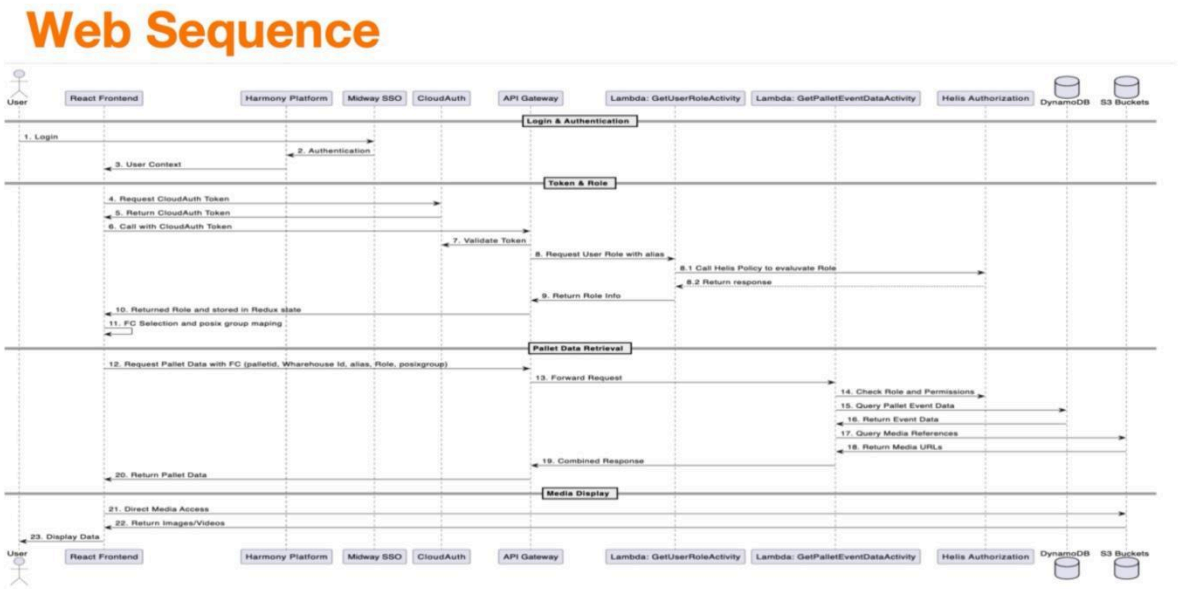

Figure 2: Web sequence from login to media display with explicit role/FC authorization checkpoints.

Login and data retrieval flow proceeds in four phases: (1) Login & Authentication via Harmony/Midway; (2) Token & Role— the client requests a CloudAuth token and then asks the service for role/FC using the user alias; (3) Pallet Data Retrieval— the client sends palletId + FC context, the service authorizes via Helis and queries DynamoDB/S3; and (4) Media Display— the UI uses returned media URLs to render images/videos with correct orientation and ordering. The design also includes a deliberate “no-access” path when role or FC checks fail.

4. Tasks and Skills Learned

4.1 Onboarding & Foundations

I completed engineering bootcamps and environment setup (Brazil build system, Cloud Desktop, IDEs), which enabled me to create pipelines and version sets for both frontend and backend. This phase taught me how large-scale organizations standardize builds, deployments, and observability for multi-team collaboration.

4.2 Frontend Evolution

I restructured the React 18 application into a cleaner component architecture while migrating to Harmony. The Harmony platform provided built-in integration points for authentication and enterprise UI patterns. On the UI side, I focused on dependable

rendering of images and videos, event timelines, barcode details, and performant filtering of large result sets. I also reviewed and addressed image rotation/cropping/ordering issues to improve analyst workflows.

4.3 Backend & APIs
I worked with Java 21 Lambda services built on the Coral framework with Smithy models and Dagger for dependency injection. For the API tier, I implemented REST endpoints behind API Gateway, evaluated CBOR versus JSON for payload efficiency, and ensured token validation and role propagation from the frontend. I also helped configure S3 resource-based access and DynamoDB queries for event metadata.

4.4 Authentication & Authorization
A key deliverable was end-to-end identity and access control. We established Midway-based authentication on Harmony for strong user identity, and implemented Helis-driven authorization using POSIX group membership and application roles (DEV, Stakeholder, FC Associate). The UI obtains user role and FC group context and includes these attributes on API requests so the service can enforce warehouse- and role-scoped permissions.

4.5 Cross-Account and Data Access
I contributed to cross-account connectivity required to call real data sources. This included IAM policy design, safe role assumption patterns, and alignment of resource policies with runtime identity attributes passed from the app.

5. Obstacles and How I Overcame Them

Role & FC scoping: The legacy tool authenticated users but treated all users equally. I evaluated several approaches (e.g., CloudAuth tokens vs. Midway cookie-based identity) and authorization backends (Helis, BRASS/Bindles). We selected Midway + Helis for secure, low-latency identity and group checks, then modeled roles and FC POSIX groups in the request lifecycle.

API format & performance: I compared CBOR and JSON. CBOR offered compact payloads and faster transmission for media-adjacent metadata. The tradeoff was ensuring both client and Lambda handlers correctly handled serialization/deserialization. I addressed this by adding format negotiation and integration tests.

Image handling: Cropping and rotation bugs were disruptive for users analyzing event sequences. I added logic for consistent orientation, predictable ordering, and efficient fetching to make side-by-side comparisons reliable.

Throttling & caching: Authorization systems can impose rate limits. I planned local caching layers and batching strategies for group lookups, and validated expected TPS envelopes.

Cross-account access: Coordinating IAM, resource policies, and security invariants across accounts required iterative testing and reviews. I wrote design notes, staged changes in beta, and validated them via integration tests and stakeholder demos.

6. Technologies and Platforms Used

• Frontend: React 18 on Harmony, TypeScript, Amazon UI components; feature-flag scaffolding for safe rollouts.
• Security: Midway SSO on Harmony; POSIX group–based authorization via Helis; least-privilege IAM and resource policies.
• API & Services: API Gateway (REST), CBOR/JSON negotiation, Java 21 Lambdas using Coral + Smithy + Dagger.
• Data Layer: DynamoDB for event metadata and S3 for media with appropriate resource-based permissions.
• DevOps: Brazil build system, version sets, pipelines; logs/metrics via platform integrations; beta-stage deployments.

7. Professional Ethics and Responsible Engineering

This project strengthened my understanding of professional ethics in software. I practiced least privilege and defense in depth, ensuring that associates only view data for their assigned FC while developers and stakeholders have appropriate, auditable access. I avoided exposing sensitive identifiers (e.g., user aliases) unnecessarily and documented design decisions for review. I also adhered to secure coding practices, prepared for data classification requirements (e.g., Harmony’s Red data roadmap), and built safe defaults—especially around authorization failures and error handling.

Deeper ethical practices applied: I treated privacy-by-design as a non-negotiable constraint: collecting only attributes needed for authorization, masking sensitive identifiers in logs, and documenting data flows for review. Transparency was balanced with safety—access-denied states explain what failed (authorization) without revealing sensitive system details.

Release discipline included staged rollouts, opt-in flags, and rollback plans. I added audit-friendly logging for role and FC checks and ensured alerts on anomalous access patterns. For vision data specifically, I considered false-positive/negative impacts and calibrated thresholds conservatively; I also reviewed edge cases (poor lighting, motion

blur) and maintained human-in-the-loop escape hatches. Finally, I aligned with retention and access-review expectations to minimize long-term exposure.

8. Management Skills Enhanced

I improved at scoping product increments, articulating tradeoffs, and meeting milestones under a fixed timeline. I kept a living plan that tracked onboarding, POCs, pipeline setup, cross-account integration, API wiring, authorization rollout, and demo deliverables. I coordinated with platform and security stakeholders, wrote a design document, and ensured that each change was testable, reversible, and well communicated.

Specific management techniques adopted: I ran a weekly rhythm of plan → build → demo, keeping a milestone plan that sequenced platform setup, identity, APIs, cross-account access, and UI fit-and-finish. I used a simple priority framework (impact × urgency) and a living dependency map (e.g., Helis group availability before API authorization tests). Risks (rate limits on auth backends, image-orientation regressions) were tracked with owners and mitigation steps. Timeboxing and acceptance criteria helped me converge decisions quickly, while short status notes shared progress, blockers, and next steps with stakeholders. Where scope shifted, I negotiated tradeoffs explicitly (for example, prioritizing IAM blockers ahead of non-critical UI polish) to keep the end-to-end slice deliverable on schedule.

9. Leadership Skills Enhanced

I proposed architectural changes (Harmony + Midway + Helis), ran POCs to de-risk unknowns, and led end-to-end demo preparation. I contributed to coding standards for the repo, documented decisions, and mentored peers on our CBOR integration and Helis permission model. Leading the demo helped me synthesize technical details into a crisp story for non-specialist audiences.

Leadership in practice: I initiated and socialized the identity/authorization design, facilitating reviews to compare alternatives and align on Midway + Helis. I proposed a concise "role × FC" contract between frontend and service, supplied reference requests/responses, and wrote a small test harness that others reused. I set lightweight coding standards (API error shapes, CBOR fallbacks, log redaction) and created a demo script covering DEV, Stakeholder, and FC Associate paths, including the deliberate "no-access" scenario. I paired with teammates to unblock CBOR serialization

issues and documented checklists/runbooks so future contributors could extend the console safely without re-learning context.

10. Outcomes and Impact

By the end of the internship, the project had:
• A Harmony-based frontend with improved component structure and a reliable image/video review experience.
• Strong authentication and fine-grained authorization that combine user role with FC POSIX group membership.
• API endpoints validating identity attributes and enforcing warehouse-scoped access.
• Cross-account paths tested against real data.
• A demo showcasing how DEV, Stakeholder, and FC Associate roles experience the console differently, including a clear “no access” case when POSIX groups do not match.

11. Future Work

Next steps include completing additional backend APIs, integrating OpenSearch for richer querying, expanding the Home page with advanced filters, adding feature flags for controlled rollouts, and extending deployments beyond the lab/beta stages.

12. Conclusion

The internship provided practical experience at the intersection of cloud platforms, security, and data-intensive UX. I strengthened my engineering fundamentals while contributing a security-focused architecture that makes the Pallet CV Console production-ready. The combination of Harmony, Midway, and Helis—along with a modern React UI and serverless backend—positions the application for reliable, scalable, and secure use across fulfillment centers.